

TECHNISCHE UNIVERSITÄT DRESDEN

FACULTY OF COMPUTER SCIENCE
INSTITUTE OF SYSTEMS ARCHITECTURE
CHAIR OF DATABASES
PROF. DR.-ING. WOLFGANG LEHNER

Diplomarbeit

zur Erlangung des akademischen Grades
Diplom-Informatiker

Active Cache-Aware Hardware Adaptation: Cache Engine for Trie-Based Index Structures

Benjamin-Elias Probst
(Born 11 April 1996 in Potsdam)

Professor: apl. Prof. Dr.-Ing. Dirk Habich

Tutor: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, June 1, 2026

Hier Aufgabenstellung einfügen!

Selbstständigkeitserklärung

Hiermit erkläre ich, dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

*Active Cache-Aware Hardware Adaptation:
Cache Engine for Trie-Based Index Structures*

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 1. Juni 2026

Benjamin-Elias Probst

Kurzfassung

Platzhalter – deutsche Kurzfassung folgt.

Abstract

Placeholder – English abstract to follow.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem Statement	3
1.3	Research Questions	3
1.4	Objectives and Contributions	4
1.5	Structure of the Thesis	4
2	Fundamentals	5
2.1	Cache Hierarchy and Cache Awareness	5
2.2	Trie-Based Index Structures	5
2.3	The Unified Comparison Interface (<code>std::map</code>)	5
2.4	YCSB Workloads	5
2.5	C++23 Metaprogramming	5
3	State of the Art	7
3.1	Trie Family (Cluster A)	7
3.2	Hybrid and B ⁺ Family (Cluster B)	7
3.3	Layout Theory (Cluster C)	7
3.4	Prefetching (Clusters D and E)	7
3.5	Synchronization (Cluster F)	7
3.6	Allocator Research (A01–A23)	7
3.7	Research Gap	7
4	Concept and Architecture	9
4.1	The Axis Library Framework	9
4.2	Four-Subsystem Separation (M Model)	9
4.3	Three-Layer Hierarchy	9
4.4	ABI-Stable C++23 Module Interface	9
4.5	The Building-Block Axes	9
4.6	PRT-ART as Probe	9
4.7	CacheEngineBuilder and Three-Stage Evaluation	9
5	Implementation	11
5.1	Three-Repository Architecture	11
5.2	Adapters for State-of-the-Art Algorithms and Allocators	11
5.3	Permutation Code Generation and Flag System	11
5.4	Concept/C RTP Realization of the Axes	11

5.5	Concurrency and Reclamation	11
5.6	Telemetry against Cache-Coherence Effects	11
5.7	Measurement Pipeline	11
6	Evaluation Methodology	13
6.1	Three Measurement Series	13
6.2	ExperimentDriver Phases	13
6.3	Hypotheses	13
6.4	Workloads and Datasets	13
6.5	Experimental Platforms	13
6.6	Permutation Explosion and Reduction	13
7	Results and Evaluation	15
7.1	Evaluation Pipeline	15
7.2	Series A: PRT-ART vs. State of the Art	15
7.3	Series B: Cache-Engine Permutations	15
7.4	Series C: Full Join	15
7.5	Axis Sensitivity Analysis	15
7.6	Discussion	15
8	Conclusion and Outlook	17
8.1	Answering the Research Questions	17
8.2	Limitations	17
8.3	Outlook	17
	Bibliography	19
	List of Figures	21
	List of Tables	23
A	Complete Measurement Series	25
B	Code Structure	27
C	Glossary	29
D	Building-Block Matrix	31
E	Architecture Decisions	33

1 Introduction

1.1 Motivation

Trie-based index structures are the backbone of modern in-memory databases. Since the Adaptive Radix Tree (ART) by Leis et al. [LKN13] in 2013, research has diversified in several directions: trie-height optimization (HOT [BZP⁺18]), hybrid B⁺ structures (Masstree [MKM12], B²-tree [?]), succinct representations (CoCo-Trie [?], SuRF [ZLL⁺18]), and self-tuning approaches (START [?]).

In parallel, a dedicated body of research on cache awareness, prefetching, and allocation strategies has emerged. Heuristics such as Cache-Sensitive Search Trees, cache-oblivious layouts, and hierarchical prefetching each address an individual axis of the cache hierarchy — yet they have never been systematically combined within a single, permuted measurement architecture.

This thesis closes that gap with a three-layer cache-engine architecture (CacheEngine → ExecutionEngine → SearchEngine) whose building blocks are decomposed into fourteen orthogonal permutation axes, spanning a configuration space of more than 10^{11} permutations. As a probe for state-of-the-art membership we contribute PRT-ART (a Probst Redirect Tree / ART hybrid) with eight building-block layers of its own.

1.2 Problem Statement

Existing cache-aware index designs optimize one or a few aspects in isolation and report results that are hard to compare across publications, because each work uses its own data structures, workloads, and measurement methodology. There is no common framework that (i) decomposes such designs into interchangeable axes, (ii) reconstructs published algorithms as explicit configurations, and (iii) measures both individual axes and whole algorithms on a single, fair basis.

1.3 Research Questions

RQ1 (Composition) Which algorithm axes of a cache-aware trie index can be canonically systematized in a permutation matrix, such that state-of-the-art implementations can be reconstructed as profile configurations?

RQ2 (Measurement methodology) How can a fair, reproducible YCSB workload run be performed per permutation, allowing profile-specific workload selection (e.g. a range-scan profile → YCSB-E) while minimizing the measurement overhead via an opt-in experiment mode?

RQ3 (Probe comparison) How does PRT-ART perform as a probe against the eight Tier-1 SOTA designs (ART, HOT, Masstree, CoCo-Trie, START, B²-tree, Wormhole, SuRF) and the Tier-2/3 SOTA under the three mandatory measurement series A (probe vs. SOTA), B (SOTA-only baseline), and C (merge points)?

1.4 Objectives and Contributions

The central contributions of this thesis are:

- A three-layer architecture (CacheEngine $\rightarrow$ ExecutionEngine $\rightarrow$ SearchEngine) with ABI-stable C++23 module interfaces and variadically templated hybrid-API specializations (1 parameter $\Rightarrow$ `std::vector` API; 2 parameters $\Rightarrow$ `std::map` API; $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>>`).
- A fourteen-axis permutation matrix with `std::variant` building blocks and compile-time fallback between the probe stack and the state-of-the-art stack.
- Thirty SOTA algorithm profiles as persisted XML configurations (8 Tier-1 + 22 Tier-2/3) that the `ExperimentDriver` transforms automatically into pre-compiled C++23 modules.
- PRT-ART as a probe with eight building-block layers of its own (4+2 pool allocator, optimistic lock coupling with reserved blocks, multi-level layout, path-oriented prefetch, density tracker, hypothesis metrics H1/H2/H3, inline/external/chain-ref value handles, signaling-bit serialization).
- A measurement driver with defined-/full-mode distinction and profile-aware workload routing.

1.5 Structure of the Thesis

Chapter 2 introduces the necessary background. Chapter 3 summarizes the state of the art across six search clusters (A–F) and an allocator cluster. Chapter 4 presents the axis library framework, the layered architecture, the ABI interface, and the fourteen-axis permutation matrix. Chapter 5 documents the implementation across the three repositories. Chapter 6 describes the measurement methodology with the three mandatory series and profile-aware workload routing. Chapter 7 presents and discusses the results, including the hypotheses H1–H3. Chapter 8 concludes with a summary and an outlook.

2 Fundamentals

2.1 Cache Hierarchy and Cache Awareness

2.2 Trie-Based Index Structures

2.3 The Unified Comparison Interface (`std::map`)

2.4 YCSB Workloads

2.5 C++23 Metaprogramming

3 State of the Art

3.1 Trie Family (Cluster A)

3.2 Hybrid and B⁺ Family (Cluster B)

3.3 Layout Theory (Cluster C)

3.4 Prefetching (Clusters D and E)

3.5 Synchronization (Cluster F)

3.6 Allocator Research (A01–A23)

3.7 Research Gap

4 Concept and Architecture

4.1 The Axis Library Framework

4.2 Four-Subsystem Separation (M Model)

4.3 Three-Layer Hierarchy

4.4 ABI-Stable C++23 Module Interface

4.5 The Building-Block Axes

4.6 PRT-ART as Probe

4.7 CacheEngineBuilder and Three-Stage Evaluation

5 Implementation

5.1 Three-Repository Architecture

5.2 Adapters for State-of-the-Art Algorithms and Allocators

5.3 Permutation Code Generation and Flag System

5.4 Concept/C RTP Realization of the Axes

5.5 Concurrency and Reclamation

5.6 Telemetry against Cache-Coherence Effects

5.7 Measurement Pipeline

6 Evaluation Methodology

6.1 Three Measurement Series

6.2 ExperimentDriver Phases

6.3 Hypotheses

6.4 Workloads and Datasets

6.5 Experimental Platforms

6.6 Permutation Explosion and Reduction

7 Results and Evaluation

7.1 Evaluation Pipeline

7.2 Series A: PRT-ART vs. State of the Art

7.3 Series B: Cache-Engine Permutations

7.4 Series C: Full Join

7.5 Axis Sensitivity Analysis

7.6 Discussion

8 Conclusion and Outlook

8.1 Answering the Research Questions

8.2 Limitations

8.3 Outlook

Bibliography

- [BZP⁺18] Robert Binna, Eva Zangerle, Martin Pichl, Günther Specht, and Viktor Leis. HOT: A height optimized trie index for main-memory database systems. In *ACM SIGMOD International Conference on Management of Data*, pages 521–534, 2018.
- [LKN13] Viktor Leis, Alfons Kemper, and Thomas Neumann. The adaptive radix tree: ARTful indexing for main-memory databases. In *29th IEEE International Conference on Data Engineering (ICDE)*, pages 38–49, 2013.
- [MKM12] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. Cache craftiness for fast multicore key-value storage. In *7th ACM European Conference on Computer Systems (EuroSys)*, pages 183–196, 2012.
- [ZLL⁺18] Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G. Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *ACM SIGMOD International Conference on Management of Data*, pages 323–336, 2018.

List of Figures

List of Tables

A Complete Measurement Series

B Code Structure

C Glossary

D Building-Block Matrix

E Architecture Decisions

Acknowledgments

Placeholder acknowledgments.

